

HW#5 Report

Domain-Specific Accelerator

楊子騫, 111550076

Abstract—本次作業實作了一個以 C 為基礎的 CNN 圖像辨識程式的 **Domain-Specific Accelerator**，透過 MMIO 將 **feature map** 與 **weight** 傳入電路，並將 **Convolution Layer** 與 **Average Pooling Layer** 的運算移至硬體實現加速電路。所使用的浮點運算 IP 包括 **FMA**、加法器、累加器、乘法器與比較器。此外，**Fully Connected Layer** 的內積運算亦移至電路中執行。最終在確保無 **Timing Violation** 且辨識正確率維持 95% 的前提下，成功達成 72.15 倍的運算加速，而在超頻 (WNS < 0) 的情況下可以達到 100 倍的加速。

Index Terms—Floating Point IP, CNN, MMIO, RISC-V, Aquila

I. INTRODUCTION

本次作業中，我將透過 MMIO 方式連接軟體與 DSA 裝置，實現資料傳入/傳出、觸發運算與等待運算等功能。加速器將負責進行 convolution layer 和 average pooling layer 的運算。Fig. 1展示了加速器的工作流程。在 convolution layer 開始時，運算參數、feature map 和 weight 會傳入 conv_layer 模組並觸發運算。完成 convolution 與 ReLU 運算後，資料會在電路中傳至 pooling_layer 模組進行 pooling 運算。完成後，資料透過 MMIO 傳回軟體。重複兩次後進入 fully connected layer，將 weight 與 neuron 透過 MMIO 傳至 calculator 模組進行內積累加運算，最後結果傳回軟體，完成數字辨識。

II. TOOL UTILIZATION

A. MMIO

我透過 Memory Map I/O (MMIO) 實現軟體與 Aquila 及 DSA 的溝通。在 Aquila 中，將 0xC400_0000 - 0xC4FF_FFFF 這段地址分配給 DSA 裝置使用。當執行 load 或 store 指令存取這段記憶體地址時，會在 soc_top.v 中選擇 DSA 裝置，並將資料的輸入與輸出接至 DSA 進行運算。

我的 DSA 裝置包含 conv_layer、pooling_layer 和浮點運算的 calculator，因此將地址範圍 0xC420_0000 - 0xC42F_FFFF 分配給 pooling_layer，0xC430_0000 - 0xC43F_FFFF 分配給 conv_layer，0xC440_0000 - 0xC44F_FFFF 分配給 calculator。這三個模組整合於 cnn_top.v，並與 soc_top.v 對接，能夠依據指定的記憶體地址在軟體中執行資料傳輸、接收、觸發運算 Busy Waiting 等操作。

B. Floating Point IP

Floating point 運算是透過 Vivado IP Catalog 中的 Floating Point IP 實現，包括 Fused-Multiply Add、Adder、Multiplier、Accumulator 和 Comparator，且均採用 non-blocking 的設計方式，避免了繁瑣的 hand-shaking 流程。

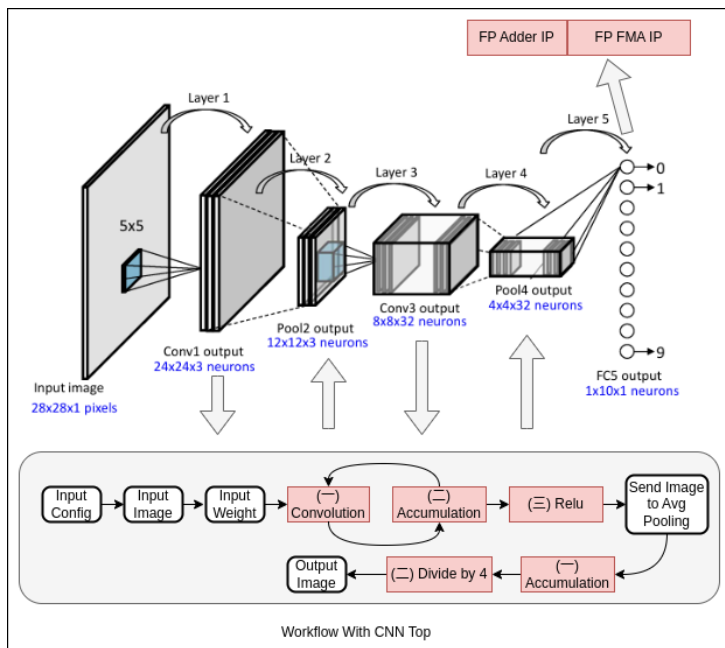


Fig. 1: CNN 加速器 Workflow

每個 IP 擁有一至三個輸入資料接口及 data valid 信號。在經過一段延遲 (latency) 後，會輸出 result valid 信號與運算結果。由於每當有一個 data valid 信號觸發時，便會生成一個 output valid 和 output data，因此在設計中，輸入資料時的 data valid 信號只會維持一個 cycle 為高，直到 output valid 出現後再傳入下一組資料，以避免運算錯誤。

IP 的 latency 可進行設定，代表輸入到輸出需經過的 cycle 數。如果將 latency 設為 0，運算會轉為組合邏輯，於 input valid 當下即輸出結果。但這會導致負的 WNS (Worst Negative Slack)，即時序延遲，表示目前的 clock period 無法支撐如此多的運算。因此，在不違反 timing constraint 的前提下，我將 Multiply 和 Comparator IP 的 latency 設為 1 cycle，其他 IP 則設為 2 cycle。

III. BASIC IMPLEMENTATION

一開始，我測試了未經修改的 CNN 軟體以及 Aquila，得到的執行時間為 21502ms，並將其作為 baseline，作為後續改進後的加速倍率參考。

A. Inner Product Calculation

首先，我嘗試將 conv3D 和 fully connected forward propagation 中的內積運算移到 DSA 進行處理，將 neuron 和 weight 傳入 DSA，並將運算結果保留在硬體中，待所有計算完成後再取出。

將 conv3D 和 fully connected layer 的內積運算改為在 IP 中執行後，執行時間縮短至 3270ms，實現 6.57 倍的加速。Fig. 2 顯示了以 fully connected forward propagation 為例的程式片段。

```
a[i] = (float_t)0;
for (uint64_t c = 0; c < entry->base.in_size_; c++){
    *((float volatile *)0xC4400018) = W[i*entry->base.in_size_ + c];
    *((float volatile *)0xC440001c) = in[c];
}
a[i] = *((float volatile *)0xC4400020);
```

Fig. 2: 內積運算的 C code

B. Preload Weight Into Hardware

經觀察發現，在捲積運算中，每張圖片會重複使用大小為 25 的 kernel。為此，我在每次處理一張圖片前，先將 25 個 weight 預載入 DSA，並於運算過程中重複使用，顯著減少了 data feeding 的時間。採用此方法後，執行時間縮短至 2872ms，實現 7.49 倍的加速。

C. Transfer All of FP Calculation

除乘法外，我發現浮點數加法也是延遲的重要來源。因此，我將 convolution layer、average pooling layer 和 fully connected layer 中的浮點數加法與乘法運算，全部改為傳入 DSA 使用 FP IP 進行計算後再傳出。這樣的改進進一步減少了執行時間，但由於計算結果尚未整理，具體加速效果需配合完整數據確認。

D. TCM Heap

我在 Linker Script 將 TCM 的剩餘所有空間（除了 boot code 之外）增設為一個 heap data section，並在 elibc 實作分配 TCM heap section 資源的 tcm_malloc 和 tcm_free，並在 CNN 軟體中將 weight 和 feature map 都使用 tcm_malloc 做分配，並且有經測試確定是使用 TCM，以達到降低 memory access latency 的效果，因為 weight 和 feature map 是主要的 memory access 目標，結合前面的方法，最後最快的運算結果是 2363ms，獲得 9.1 倍的加速。不過這個優化因為後面將大部分運算轉為電路運算，降低 memory access 的次數，所以在後面的加速效果變得微乎其微。

IV. ADVANCED IMPLEMENTATION DETIAL

從前一個小節可以看出，單純將浮點數運算改為使用 FP IP 的加速效果有限，嘗試了很多細節的調整，最高也只有到 9.1 倍的加速，若要有階段性的提升，就要把原本是 C base 架構的 convolution 和 pooling 計算直接改造成電路計算，才有辦法突破計算的 bottleneck。Fig. 3 是最終的系統架構圖。接下來會進一步說明每一個 module 的實作細節。

A. Convolution Layer

最一開始先將參數、feature map、weight 都透過 MMIO 的方式傳入 conv_layer module，使用 register 儲存，將 ram style 設為"block"，並且簡化 IO 的邏輯，才可以讓較大數量的 register 使用 block ram 合成，降低 LUT 的使用。接著在軟體 trigger 計算，並且使用 busy waiting 的方式等待硬體計算完畢。

開始計算時先用 Fused-Multiply Add IP 計算一次捲機運算，接著將計算結果使用 Adder IP 累加到 output_img register，以這樣的方式將 In Channel 的每一張圖片的捲機結果做累加。最後是

RELU 的步驟，使用 Comparator IP，和 0 做比較，若結果是比 0 小就將那個點設為 0。全部完成後，再將 output_img register 的值透過 FSM 控制 strobe 信號，傳至 pooling_layer module 並寫入 pooling layer 的 register，並將 input size 傳過去。整個過程使用 Finite State Machine 控制流程。

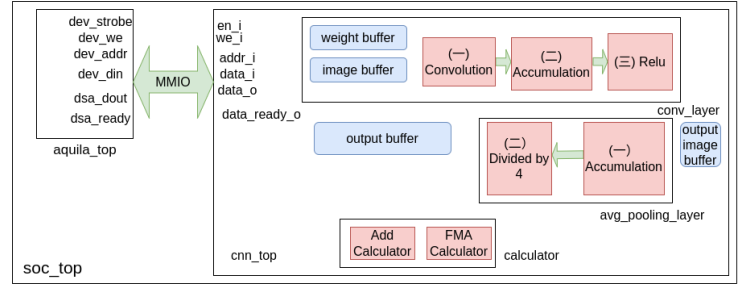


Fig. 3: CNN 加速器系統架構圖

```
uint64_t total_size = entry->base.out_size_;
*((int volatile *)0xC430000c) = 1; // trigger calculation.
while(*((int volatile *)0xC4200004) == 0);
for(int i = 0; i < total_size; i++)
{
    out[i] = *((float volatile *)0xC4200000);
}
```

Fig. 4: 觸發並等待計算、回傳資料的 C code

B. Pooling Layer

從 convolution layer 傳入 feature map 後，透過 index 的計算，先使用 Accumulator IP 將 kernel 內的四個點累加起來，當累加到第四個點時，將 tlast 設成 1，一樣是只會維持一個 cycle，這樣當 result_tlast 變成 1 的時候，就是四個數值的累加結果。接下來再用 Multiply IP 將結果乘以 0.25，達到 average pooling 的效果，整個實作一樣是使用 Finite State Machine 控制流程。運算完成後解除軟體的 busy waiting，並透過 MMIO 的方式將運算結果傳回到軟體，Fig. 4 是觸發計算並等待結果，最後回傳 feature map 的 C code。

C. Fully Connected Layer (Ver. 1)

將 fully connected layer 的內積運算改為 Fused-Multiply Add IP 運算，並將 bias 運算改為使用 Adder IP 運算，並使用 MMIO 做輸入/輸出資料。

D. Fully Connected Layer (Ver. 2)

實際上我也有實做 fully connected layer 的電路運算，步驟是先使用 Fused-Multiply Add IP 做內積計算，接著使用 Adder IP 做 bias 計算，然後是 RELU，和前面一樣使用 Comparator IP 和 0 做比較，若結果小於 0 就設成 0。最後是另一個 Comparator IP，使用類似軟體線性找最大值得方式，從 index=0~9 來比較運算結果並儲存最大值和 index，最後可以直接輸出預測結果給軟體 (0-9 的整數)，但是最後的總運算時間意外的比只將內積運算和 bias 加法改為 IP 運算的方法還慢，下一個章節會進一步比較兩種方法的花費時間。

TABLE I: 不同方法的加速效果及 WNS 比較

Method	Original	Basic Method	Fully Layer Ver. 2	Fully Layer Ver. 1	Fully Layer Ver. 1	Fully Layer Ver. 1
Aquila Clock Rate	50MHz	50MHz	50MHz	50MHz	50MHz	28MHz
FMA Latency		2	2	2	1	1
spend time(ms)	21502	2363	338	298	215	393
speedup	1	9.1	63.62	72.15	100.01	54.71
WNS	>0	>0	>0	>0	-13.323	>0

TABLE II: 傳輸資料及計算的耗時分析

Layer	Convolution Layer				Pooling Layer				
Type	Load Image	Load Weight	FMA CALC	Send Out Img	ACCUM CALC	MUL CALC	Send Out Img	Other	total
Cycle	383812	346769	12122880	75560	56680	113508	132440	1668351	14900000
Spend Time(ms)	7.67624	6.93538	242.4576	1.5112	1.1336	2.27016	2.6488	33.36702	298
Ratio	2.58%	2.33%	81.36%	0.51%	0.89%	0.38%	0.76%	11.20%	100%

TABLE III: 硬體資源使用分析

Note	Name	Slice LUTs	Slice Registers	F7 Muxes	F8 Muxes	Slice	LUT as Logic	LUT as Memory	Block RAM Tile	DSPs
	Maximum	63400	126800	31700	15850	15850	63400	19000	135	240
Before Optimization	conv_layer	12554	1008	1796	193	3544	3466	9088	5	10
After Optimization	conv_layer	1435	813	4	1	591	1435	0	11	10
	pooling_layer	1254	346	1	0	472	893	352	2	5
	calculator	1023	584	4	1	337	1023	0	0	8

V. EVALUATION

A. Speed Up and Timing Constrains

TABLE 1是不同方法的加速效果及 WNS 比較，其中”Fully Layer Ver. 2”就是前面提到的將整個 fully connected layer 直接變成電路計算的版本，”Fully Layer Ver. 1”是只將最後一層的內積運算改成 IP 運算的最終版本，最後在作業時間內沒有找出 Ver. 2 比較慢的原因，寒假會再研究這個部份。

我嘗試將 Fused-Multiply Add IP 的 latency 降至 1，雖然可以正確運行，正確率維持 95%，且速度可以升至 100.01 倍，但是 Implementation Design 的 timing report 中，Setup time 的 WNS (Worse Negative Slack) 為-13.323，說明在 50MHz 的 clock rate 下，無法在一個 cycle 的 latency 內做完 FMA 運算，需要降低 clock rate 才不會超頻。

我有嘗試將 Aquila 的 clock rate 降低來嘗試讓 FMA 的 latency 可以是 1 cycle，但我降到 28MHz 才讓 WNS 恢復正常，不過這時的速度因為 clock rate 降低而降到 54.71 倍，因此我最後提交的 code 以及以下分析的部份是使用 FMA latency 為 2 的最終版本，花費時間為 298ms，有 72.15 倍的加速效果。

B. Overhead Analysis

TABLE 2是最終版本的熱點分析，分析方法是使用 counter 計算 conv_layer 和 pooling_layer 中的 Finite State Machine 不同狀態停留時間。從表中可以看出，最大的 bottleneck 還是在內積計算，

佔有 81.36% 的時間，因為要對每張圖的每 25 個位置做乘法累加，是很大的計算量，因此卷積運算是最主要的優化目標，至於其他像是 Pooling Layer 運算就佔很少的比例。

傳輸資料的部份也佔非常少的時間，全部只佔 5-6% 左右，另外還有 11% 左右的時間是在 fully connected layer，也可以再稍作優化。不過最主要的 bottleneck 還是在卷積運算。

C. FPGA Resource Utilization

TABLE 3 顯示了 FPGA 硬體資源的使用情況。從表中可以看出，convolution layer 和 pooling layer 均成功利用了 Block RAM 資源。特別是在 convolution layer 中，初始設計中由於 output image register 寫法過於複雜，導致 LUT as Memory 和其他邏輯單元的使用量非常高，基本上是無法放入第二個 conv layer。

為了解決這一問題，我參考了 AMD 官方網站的建議，採用了基於 Block RAM 的實現方法，使該暫存器能夠被合成為 Block RAM。優化後的結果顯示，conv_layer 的 LUT as Memory 使用量降為 0，而 Block RAM 的使用量增加至 11。同時，其他邏輯單元的資源使用量也顯著下降，充分說明了此優化的必要性與有效性。

此外，從表中的 ARTY-A7 100T 資源使用 Maximum 可以看到，Block RAM 和其他邏輯單元的使用量均非常充足，為未來增加圖片尺寸、通道數 (Channel) 以及模型複雜度提供了充裕的擴展空間。

VI. FUTURE WORK

A. Integration

因為這次作業是一層一層慢慢寫的，沒有為整合作設計，因此每兩層都需要軟體輔助才能進行，但其實我的所有運算都已經寫成電路，可以從一開始 load 進去 image 和 weight 後就透過電路輸出結果，因此若有時間我會嘗試將整個電路整合為一個 CNN 電路，而不用靠軟體輔助。

B. Scalability

因為這次作業沒有 padding，stride 是 1，且實際上沒有 bias，所以我實際上沒有處理這三個運算，另外我也沒有額外將 width 將 height 分別處理，而是都以 width 平方的方式在計算數量，沒有考慮長寬不同大小的情況。因此若要擴展為廣泛使用的 project，需要這些擴展性的強化。

C. Quantization

Quantization 是將 weight 和 Image 都轉為 8 bit 整數的數學原理，由於 Image 本身是 8 bit 的灰階數值，因此只要將 weight 也轉為 8 bit 整數，並且確保不會降低 accuracy，就可以透過將四個 8 bit 數據包成一個 32 bit 的資料的方式，大幅提升傳輸資料時間，而在 DSA 的運算也可以變成比較簡單的直接乘法運算，不用使用一大堆浮點數 IP，和相對應的複雜設計。因此我認為混合精度是可以嘗試的方向。

VII. CONCLUSION

這次作業我是一步一步從最基本的方法，慢慢改進更多部份，從 6.5 倍一路提升到 72 倍，是非常有成就感的過程，之前沒有嘗試過將這麼大範圍且複雜的計算用電路寫出來的題目，一開始也是用非常暴力的方法，後來慢慢抓到控制的技巧，才回頭將一些很暴力的部份改進，也學到了很多，像是 Block Ram 有規定的寫法、如何處理超頻等都是在這次作業中才實作的。希望寒假可以將這個 project 整理並改進成一個完整且可擴展的專案。